

Compiler Design — CST302

Module 3: Bottom-Up Parsing

Topics covered: Bottom-Up Parsing Fundamentals · Handles & Handle Pruning · Shift-Reduce Parsing · Operator Grammar & Operator Precedence Parsing

1. Bottom-Up Parsing Fundamentals

Definition

A **Bottom-Up Parser** starts from the **input string (terminals)** and tries to **reduce it to the start symbol** using grammar productions in reverse. The parse tree is built from **leaves to root**.

Flow:

Input String → Reduce step by step → Start Symbol

Example Grammar

$E \rightarrow E + T \mid T$
 $T \rightarrow T * F \mid F$
 $F \rightarrow (E) \mid \text{id}$

Input: **id * id**

Reduction steps:

Step	String	Action
1	id * id	Start
2	F * id	Reduce id → F
3	T * id	Reduce F → T
4	T * F	Reduce id → F
5	T	Reduce T * F → T
6	E	Reduce T → E ✓ Accept

2. Top-Down vs Bottom-Up Parsing

Feature	Top-Down Parsing	Bottom-Up Parsing
Direction	Start Symbol → Input	Input → Start Symbol
Tree construction	Root → Leaves	Leaves → Root

Derivation used	Leftmost derivation	Rightmost derivation in reverse
Example parsers	Recursive Descent, LL(1)	Shift-Reduce, LR

3. Handle

Definition

A **handle** is a **substring of the input that matches the RHS of a production rule** and whose reduction represents **one step of reverse rightmost derivation**.

Example

Grammar:

$E \rightarrow E + E \mid E * E \mid (E) \mid id$

Input: **id + id * id**

Rightmost derivation:

$E \rightarrow E + E \rightarrow E + E * E \rightarrow E + id * id \rightarrow id + id * id$

Right Sentential Form	Handle	Reducing Production
id + id * id	id	$F \rightarrow id$
F + id * id	id	$F \rightarrow id$
F + F * id	id	$F \rightarrow id$
F + F * F	F * F	$E \rightarrow E * E$
F + E	F + E	$E \rightarrow E + E$
E	—	Accept

4. Handle Pruning

Handle pruning is the process of **obtaining the rightmost derivation in reverse order by repeatedly identifying and reducing handles**.

Example — Input: id + id * id

```
id + id * id
↓ reduce id → E
E + id * id
↓ reduce id → E
E + E * id
↓ reduce id → E
E + E * E
↓ reduce E * E → E
E + E
↓ reduce E + E → E
```

E ✓ (Start Symbol reached)

5. Shift-Reduce Parsing (SRP)

Shift-Reduce Parsing is the **most common bottom-up parsing technique**. The parser uses a **stack** and an **input buffer** to simulate rightmost derivation in reverse.

Initial and Final Configurations:

State	Stack	Input
Initial	\$	w \$
Final (Accept)	\$ S	\$

Four Actions of Shift-Reduce Parser

Action	Description	Example
SHIFT	Move next input symbol onto the stack	$\$ \rightarrow \$ \text{id}$
REDUCE	Replace RHS on stack with LHS non-terminal	$\$ \text{id} \rightarrow \$ F$ (using $F \rightarrow \text{id}$)
ACCEPT	Announce successful parse completion	Stack: $\$S$, Input: $\$$
ERROR	Syntax error detected; call error routine	Invalid token sequence

Example: Stack Trace for id + id

Grammar: $E \rightarrow E + T \mid T, T \rightarrow \text{id}$

Input: id + id

Stack	Input	Action
\$	id+id\$	Shift
\$ id	+id\$	Reduce $T \rightarrow \text{id}$
\$ T	+id\$	Reduce $E \rightarrow T$
\$ E	+id\$	Shift
\$ E +	id\$	Shift
\$ E + id	\$	Reduce $T \rightarrow \text{id}$
\$ E + T	\$	Reduce $E \rightarrow E+T$
\$ E	\$	Accept ✓

6. Operator Grammar

A grammar is an **Operator Grammar** if it satisfies **both** of the following conditions:

- **No ϵ -productions** (no empty RHS in any production)
- **No two adjacent non-terminals** on the RHS of any production

Grammar	Operator Grammar?	Reason
$E \rightarrow E+E \mid E^*E \mid (E) \mid \text{id}$	✓ Yes	No ϵ , no adjacent non-terminals
$E \rightarrow EOE, A \rightarrow + - * /$	✗ No	$E O E$ has two adjacent non-terminals (E, O, E)
$A \rightarrow BC$	✗ No	B and C are adjacent non-terminals

7. Operator Precedence Relations

Operator precedence parsers use **three relations** between terminals to find handles:

Relation	Notation	Meaning
Less Than	$a \ \<\cdot\ b$	a yields precedence to b (b has higher precedence)
Equal	$a \ =\cdot\ b$	a has the same precedence as b
Greater Than	$a \ \cdot>\ b$	a takes precedence over b (b has lower precedence)

Example — Input: $\text{id} + \text{id} * \text{id}$

Precedence table (for grammar $E \rightarrow E+E \mid E^*E \mid \text{id}$):

String with relations inserted:

$\$ \ \<\cdot\ \text{id} \ \cdot>\ + \ \<\cdot\ \text{id} \ \cdot>\ * \ \<\cdot\ \text{id} \ \cdot>\ \$$

Interpretation: $*$ has higher precedence than $+$, so $\text{id} * \text{id}$ is reduced first.

8. Complete Concept Flow (Summary)

Concept	Key Idea	Exam Importance
Bottom-Up Parsing	Input $\rightarrow$ Start Symbol (leaves to root)	High
Handle	Substring matching RHS of a production	High
Handle Pruning	Repeated reduction = reverse rightmost derivation	High
Shift-Reduce Parsing	4 actions: Shift, Reduce, Accept, Error	Very High
Operator Grammar	No ϵ , no adjacent non-terminals on RHS	Medium
Operator Precedence	3 relations: $\<\cdot, =\cdot, \cdot>$ between terminals	Medium